

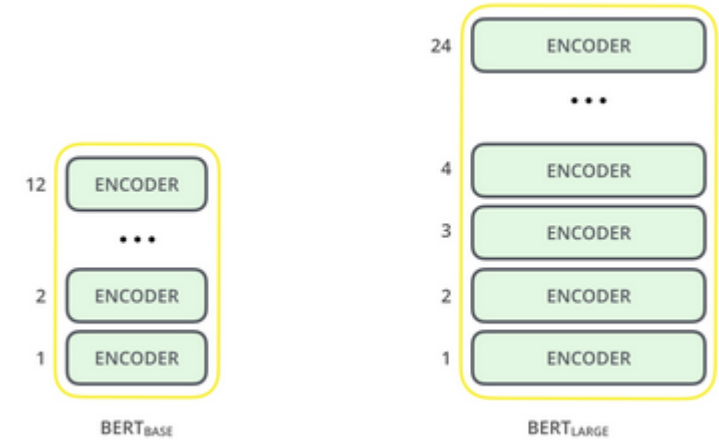
Encoder only Transformer models: Bert

Bidirectional Encoder Representation from Transformer

BERT

- BERT stands for **Bidirectional Encoder Representations from Transformers**.
- It is a **transformer-based language model** developed by Google in 2018 that became a foundation for many modern NLP applications.
- BERT is based purely on the **Transformer Encoder** (not the decoder part).
- It is pre-trained on a large corpus of unlabelled text including the entire Wikipedia(2,500 million words!) and Book Corpus (800 million words).
- The key strength of BERT lies in its **ability to understand language bidirectionally**, using information from both the left and right contexts of a word during the training phase.
- As a result, the pre-trained BERT_model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of NLP tasks.

- A typical version, **BERT-base**, consists of:
 - 12 transformer encoder layers (also called transformer blocks)
 - Hidden size of 768 (embedding dimension)
 - 12 self-attention heads
 - 110 million trainable parameters
- The larger variant, **BERT-large**, has 24 encoder layers, hidden size of 1024, 16 attention heads, and 340 million parameters.



	BERT _{BASE}	BERT _{LARGE}	Transformer
Layers	12	24	6
Feedforward networks (hidden units)	768	1024	512
Attention heads	12	16	8

BERT Architecture Components

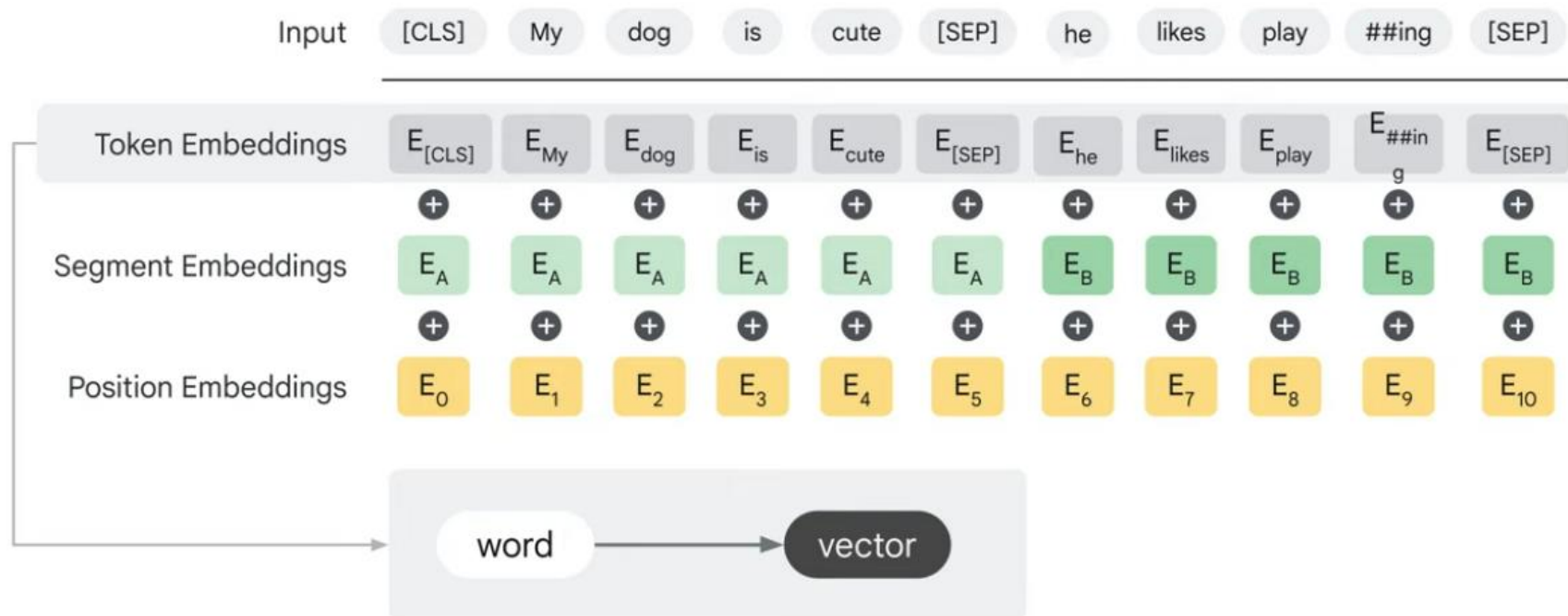
- Input Embeddings
- Transformer Encoder Stack
- Pretraining Objectives
- Fine tuning

1. Input Representation

- Before entering the transformer blocks, every input sentence is converted into embeddings that combine three types of information:
- **Token embeddings:** Each word is tokenized into subwords using the WordPiece tokenizer, and each token is mapped to a vector.
- **Position embeddings:** Since transformers have no inherent sense of word order, positional embeddings are added to preserve sequence information.
- **Segment embeddings:** When BERT receives a sentence pair (e.g., for question answering or entailment), tokens from the first sentence are assigned *Segment A embeddings*, and tokens from the second are assigned *Segment B embeddings*. For single sentences or paragraphs, only Segment A is used.
- These three embeddings are summed to form the input vector for each token. Special tokens are also used:
 - [CLS] is placed at the beginning, and its final hidden state is often used for classification tasks.
 - [SEP] is used to mark the end of a sentence or separate two sentences.

- Final input vector of a given token (word) = Token + Segment + Position embedding.
- A [CLS] token is added to the beginning of the first sentence to indicate the beginning of a sentence, and a [SEP] token is added to the end of each sentence to mark the end of a sentence.

BERT input embeddings



2. Transformer Encoder Blocks

Each encoder block in BERT is composed of two main layers:

1. Multi-Head Self-Attention

1. This mechanism allows every token to attend to every other token in the sequence, both before and after it (bidirectional).
2. Multiple attention heads are used, each learning different types of relationships (e.g., syntactic, semantic).
3. The outputs of all heads are concatenated and linearly projected.

2. Feed-Forward Neural Network (FFN)

1. Each token representation is passed through a two-layer fully connected network with a non-linear activation (usually GELU- Gaussian Error Linear Unit).

Both layers are wrapped with **residual connections** and **layer normalization**, which stabilize training and help gradient flow. Dropout is applied for regularization.

3. Pre-training BERT

- Why Pretraining?
 - BERT is trained on massive unlabeled text (Wikipedia + BookCorpus). It learns general language understanding before task-specific fine-tuning.
 - Pretraining BERT reduces the dependence on large labeled datasets because the model already knows general language patterns from unlabeled data.

- **Pretraining Objectives**

- BERT uses two self-supervised tasks:

- **(a) Masked Language Modeling (MLM)**

- MLM allows BERT to **learn bidirectional contextual representations**.
 - Unlike left-to-right models (GPT), it can understand the **whole sentence context** (both before and after a token). This is why BERT embeddings are powerful for classification, QA, NER, etc.
 - For this, BERT randomly mask **15% of tokens** in input.
 - Then the model is trained to predict the original tokens from context (both left & right).
 - Example:
 - Input: *"The cat [MASK] on the mat."*
 - Output: *"sat"*.

- **(b) Next Sentence Prediction (NSP)**

- Task: Given a pair of sentences, the model predicts whether the second sentence follows the first in the original text. This trains BERT to understand sentence-level relationships.
 - 50% positive examples, 50% random negatives.
 - Example:
 - Sentence A: *"I went to the market."*
 - Sentence B: *"I bought fresh apples."* → **IsNext**.
 - Sentence B: *"The moon is bright tonight."* → **NotNext**.

- Through these tasks, BERT learns rich, contextualized embeddings that can be adapted to many NLP tasks.

4. Fine-Tuning

- After pretraining, BERT can be fine-tuned with minimal task-specific modifications.
- A small classification or prediction layer is added on top, and the entire model is trained end-to-end on the downstream dataset.
- This simple yet powerful approach allows BERT to achieve state-of-the-art results on tasks such as sentiment analysis, question answering, named entity recognition, and natural language inference.

Decoder-Only Models in Transformers

What Are Decoder only models?

- In the original **Transformer** architecture (Vaswani et al., 2017):
 - The **encoder** reads the source sequence (e.g., English sentence).
 - The **decoder** generates the target sequence (e.g., French translation), using both self-attention and cross-attention to the encoder outputs.
- A **decoder-only model** removes the encoder part. It keeps only the **stack of decoder blocks** but with one important simplification:
 - **Cross-attention is dropped** (since there is no encoder to attend to).
 - Only **causal self-attention (masked attention)** is used (tokens can only attend to previous tokens, not future ones).
- This makes the model **autoregressive**: it generates one token at a time, conditioning only on the past.

- **Significance and Applications**

- GPT's architecture demonstrates that a **decoder-only transformer trained at scale** can serve as a universal language model. It excels at:
 - Text completion and story generation
 - Conversational agents
 - Summarization
 - Code generation
 - Knowledge extraction and reasoning
- Its flexibility makes GPT a foundation for modern large language models, inspiring a wave of successors like LLaMA, Falcon, and Gemini.

Structure of decoder-Only Models

- Each decoder block has:
 - **Masked Multi-Head Self-Attention**
 - Attention is masked so a token cannot see future tokens.
 - Ensures left-to-right generation.
 - **Residual connections + LayerNorm**
 - Stability and gradient flow.
 - **Feed-Forward Network (FFN)**
 - Nonlinear transformation of token representations.
 - No encoder–decoder cross-attention layer is present.

- **Key Examples of Decoder-Only Models**
- Almost all modern large language models (LLMs) are decoder-only:
 - GPT family (GPT-1, GPT-2, GPT-3, GPT-4, GPT-5)
 - LLaMA (Meta)
 - Falcon
 - Mistral
 - Gemini (Google)
 - Claude (Anthropic)

What Are They Good For?

- Decoder-only models excel at **generation tasks**, because they predict the next token given the history.
 - Next-word prediction (language modeling)
 - Text completion
 - Dialogue/chatbots
 - Code generation
 - Story writing, summarization, translation (without explicit encoder)
- They are less natural for **classification** or **sequence labeling** tasks, but can still be adapted by prompting or fine-tuning.

GPT

- Generative Pretrained Transformer (GPT) is one of the most significant architectures in modern natural language processing.
- Developed by **OpenAI**, it represents a shift from task-specific models toward large-scale, general-purpose models capable of generating and understanding language with minimal supervision.
- GPT is built entirely on the **transformer decoder** architecture and is trained in an autoregressive manner to predict the next word in a sequence.
- Its ability to generate coherent, contextually appropriate text stems from its architecture and training paradigm.

Architecture

- GPT is a **decoder-only transformer** consisting of multiple stacked layers (or blocks).
- Each layer is responsible for progressively refining the representation of the text.
 - For example, GPT-2 small has 12 layers, while GPT-3 uses 96 layers with 175 billion parameters.
- The general structure of each block includes:
 - **Masked Multi-Head Self-Attention**
 - **Feed-Forward Neural Network (FFN)**
 - **Residual Connections and Layer Normalization**
- This design allows GPT to model long-range dependencies in text while preserving training stability.

Input Representation

- Before text enters the transformer blocks, it must be converted into a numerical form. GPT achieves this using two key embedding types:
 - **Token Embeddings:**

GPT uses the **Byte Pair Encoding** (BPE) tokenizer, which breaks words into subword units. Each subword is mapped to a dense vector, capturing semantic and syntactic properties.
 - **Positional Embeddings:**

Unlike recurrent models, transformers do not have a natural sense of order. Positional embeddings encode the position of each token in the sequence, ensuring that the model can distinguish between “dog bites man” and “man bites dog.”
- The final input representation is obtained by summing token embeddings and positional embeddings. Unlike BERT, GPT does not use segment embeddings, as it typically processes single sequences.

Core Components

- **(a) Masked Multi-Head Self-Attention**

- The heart of GPT is its **self-attention mechanism**, which enables each token to attend to all previous tokens in the sequence.
- However, GPT uses a **causal (triangular) mask**, ensuring that a token cannot see future tokens. This maintains the autoregressive property—predicting the next word based only on the past.
- Multi-head attention involves projecting the input embeddings into multiple subspaces (queries, keys, and values) and computing attention scores.
- Different heads capture different types of relationships, such as syntactic dependencies or semantic similarities.

- **(b) Feed-Forward Neural Network (FFN)**

- After self-attention, each token representation is passed through a position-wise feed-forward network.
- This is typically a two-layer fully connected network with a nonlinear activation function (such as GELU). It helps transform the attended information into richer feature representations.

- **(c) Residual Connections and Layer Normalization**

- To stabilize training and improve gradient flow, GPT employs residual (skip) connections around both the self-attention and feed-forward sublayers.
- After adding the input back to the output of each sublayer, **layer normalization** is applied. This ensures that representations remain numerically stable across many stacked layers.

- **Pretraining Objective**

- GPT is trained using the **autoregressive language modeling objective**. Given a sequence of tokens:

The cat sat on the

the model is tasked with predicting the next token:

mat

Formally, GPT maximizes the likelihood:

$$P(w_t | w_1, w_2, \dots, w_{t-1})$$

The training uses **cross-entropy loss** over the predicted vocabulary distribution. By doing this at scale, GPT learns to capture grammar, semantics, and even world knowledge.

- **Text Generation**

- At inference time, GPT generates text sequentially:
 - Start with a prompt (e.g., “Once upon a time”).
 - Predict the next token distribution.
 - Sample or select the most likely token
 - Append the token to the sequence and repeat until a stopping condition is reached.
- This process allows GPT to generate coherent paragraphs, stories, code, or answers in a conversational setting.

- **Fine-Tuning and Adaptation**

- Although pretrained on general corpora, GPT can be fine-tuned for specific applications.
 - Fine-tuning involves continuing training on task-specific data, adjusting the weights slightly while retaining the pretrained knowledge.
 - More recently, GPT has been adapted using **Reinforcement Learning from Human Feedback (RLHF)** to align outputs with human preferences, leading to conversational models such as ChatGPT.
- Note: RLHF = **pretrain LM → supervised fine-tune → reward model → RL optimization** to align models with human preferences

Appendix

How BERT is trained bidirectionally

◆ 1. Start with a large text corpus

- BERT was originally trained on:
 - **BooksCorpus** (800M words)
 - **English Wikipedia** (2.5B words, excluding lists, tables, headers)
 - Any large, raw, unlabeled text corpus works.
-

◆ 2. Tokenization (WordPiece)

- Raw text → tokenized into subwords (WordPiece vocab, ~30k tokens).

Example:

vbnet

Sentence: "Understanding transformers is amazing."

Tokens: [CLS] understanding transform ##ers is amazing . [SEP]

◆ 3. Create sentence pairs (for NSP)

Each training example is a pair of sentences:

1. Positive pair (IsNext):

Take two consecutive sentences from the corpus.

CSS

Sentence A: "The cat sat on the mat."

Sentence B: "It was purring softly."

Label: IsNext (1)

2. Negative pair (NotNext):

Take Sentence A, and pair it with a random sentence from elsewhere.

CSS

Sentence A: "The cat sat on the mat."

Sentence B: "The stock market crashed yesterday."

Label: NotNext (0)

- Half of the training pairs are **IsNext**, half are **NotNext**.

◆ 4. Apply masking (for MLM)

Now apply **15% masking strategy** to tokens in the combined sequence:

- Suppose the pair is:

vbnet



Input: [CLS] The cat sat on the mat . [SEP] It was purring softly . [SEP]

- Randomly select tokens (e.g., "sat", "mat", "purring") → apply rules:
 - 80% → replace with [MASK]
 - 10% → replace with a random token
 - 10% → keep unchanged

less



Masked Input: [CLS] The cat [MASK] on the [MASK] . [SEP] It was [MASK] softly . [SEP]

Labels: - - sat - mat - - - purring - -

- The model sees the **masked input** but is trained to recover the original tokens.

◆ 5. Add segment embeddings + position embeddings

- Sentence A tokens → Segment ID 0
- Sentence B tokens → Segment ID 1
- Positional encodings → maintain order.

This helps the transformer know which tokens belong to which sentence.

✅ Final Training Example

Input IDs (with masking, WordPiece tokens)

CSS

[CLS] The cat [MASK] on the [MASK] . [SEP] It was [MASK] softly . [SEP]

Segment IDs

0 0 0 0 0 0 0 0 1 1 1 1 1 1

Attention mask (1 for real tokens, 0 for padding if needed).

Labels

- MLM labels: sat, mat, purring
- NSP label: IsNext (1)

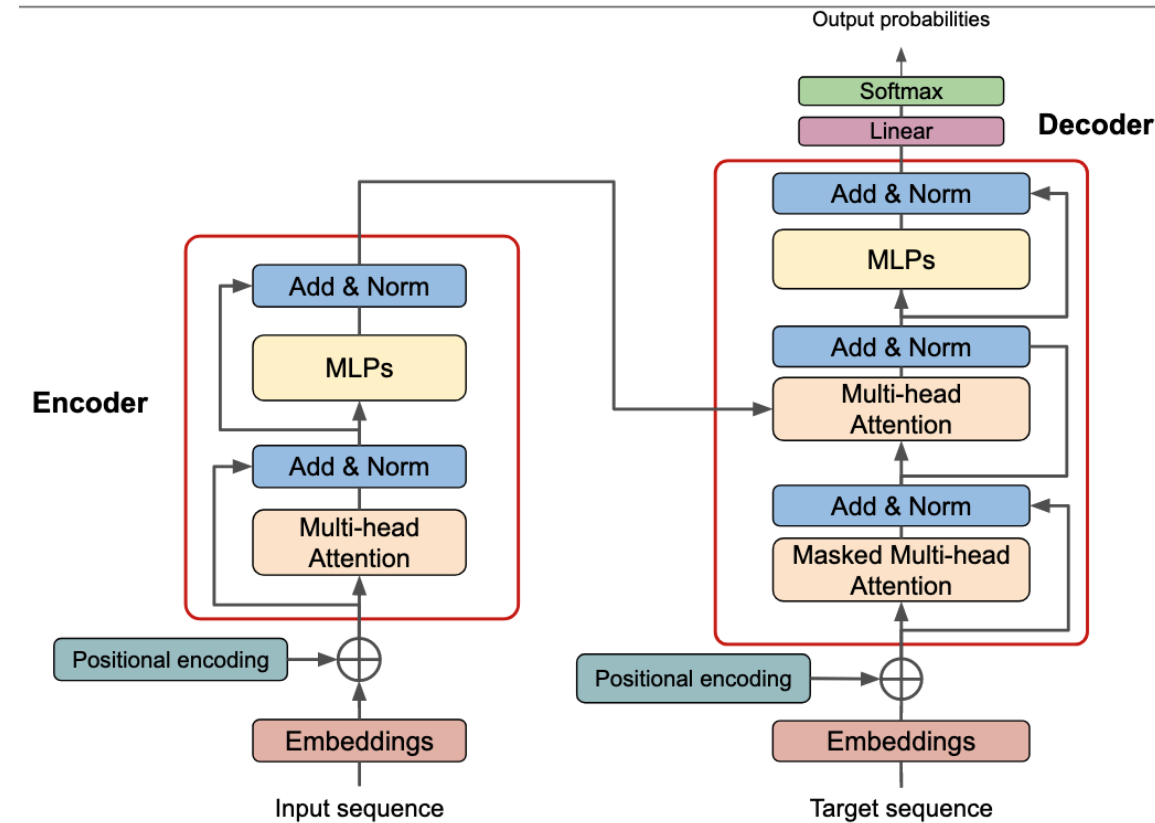
By preparing the dataset this way, each token's prediction **requires looking left & right simultaneously**, making BERT truly **bidirectional**.

[Transformer models and BERT model: Overview \(youtube.com\)](#)

[BERT Neural Network - EXPLAINED! \(youtube.com\)](#)

Introduction

- The transformer is a neural network architecture which uses **self-attention mechanisms** to focus on different parts of the input text during each processing step.
- A transformer model captures more context and improves performance on natural language processing (NLP) tasks.
- **Overall Structure**
 - **Encoder** (stack of N identical layers)
 - **Decoder** (stack of N identical layers)
 - **Final output layer** (usually a linear layer + softmax for language tasks)



- **Inside Each Encoder Layer**

- 1. Input Embedding Layer**

1. Converts tokens into dense vectors.

- 2. Positional Encoding**

1. Adds position information to embeddings.

- 3. Multi-Head Self-Attention**

1. Allows each position to attend to all others in the sequence.

- 4. Add & Norm**

1. Residual connection + Layer normalization after attention.

- 5. Feed-Forward Network (FFN)**

1. Two fully connected layers with a non-linear activation (ReLU/GELU).

- 6. Add & Norm**

1. Residual connection + Layer normalization after FFN.

Inside Each Decoder Layer

In addition to encoder components:

- 1.Masked Multi-Head Self-Attention**

1. Prevents attending to future tokens (causal masking).

- 2.Multi-Head Cross-Attention**

1. Allows decoder to attend to encoder outputs.

3.Add & Norm layers after each attention block and FFN.